

UNIVERSITY OF NEUCHÂTEL

Master in Computer Science

**Reinforcement Learning and Decision Making
Under Uncertainty**

SPRING SEMESTER 2026

Final Project

Report on

**From Heuristics to Reinforcement Learning :
Benchmarking Q-Learning, DQN-MLP and PPO
on the Snake Game**

Professor: Christos Dimitrakakis

Teaching Assistant: Jakub Tluczek

Students: Allizha Theiventhiram
Boris Verdecia Echarte

May 29, 2026

Contents

1	Introduction	3
2	Problem Formulation	3
2.1	MDP Definition	3
2.2	Reward Function Design	4
3	Environment Implementation	4
3.1	Gymnasium Wrapper	4
4	Algorithms and Implementation	5
4.1	Random Baseline	5
4.2	Tabular Q-Learning	5
4.3	Deep Q-Network (DQN-MLP)	5
4.4	Proximal Policy Optimization (PPO)	6
5	Experiment Design	7
5.1	Evaluation Protocol	7
5.2	Metrics	7
5.3	Reproducibility	7
6	Training	8
6.1	Q-Learning	8
6.2	DQN-MLP	9
6.3	PPO	10
6.4	Convergence Summary	10
7	Results	11
7.1	Sparse vs. Dense Reward	11
7.2	Grid Size Ablation	12
7.3	Obstacles Ablation	13
8	Discussion	14
9	Conclusion	15
10	Use of AI Tools	15

Abstract

We present a comparative study of reinforcement learning algorithms applied to the Snake game, a discrete grid-based control problem modelled as a Markov Decision Process. We benchmark three algorithms of increasing complexity — tabular Q-Learning, Deep Q-Network with an MLP (DQN-MLP), and Proximal Policy Optimization (PPO) — across three ablation dimensions: reward function (sparse vs. dense), grid size (8×8 to 20×20), and environment complexity (with and without obstacles). All evaluations are conducted over 5 independent seeds with bootstrapped 95% confidence intervals. Results confirm a clear performance hierarchy on the base setting (sparse, 8×8 , no obstacles): Random < Q-Learning < DQN-MLP < PPO. Dense reward shaping improves Q-Learning and DQN-MLP but surprisingly degrades PPO. Grid size ablation reveals that Q-Learning and DQN-MLP generalise better than PPO to larger unseen grids, where they eventually surpass it. Finally, all algorithms degrade under obstacles, with PPO showing the largest score drop despite maintaining the highest survival time.

1 Introduction

The Snake game is a discrete control problem where an agent moves on a grid, eats food to grow, and must avoid hitting walls or its own body. It is a useful testbed for reinforcement learning because the difficulty scales naturally with grid size, credit assignment is non-trivial (early decisions affect survival much later), and the agent must balance short-term rewards with long-term survival [1].

This project builds on prior work from our Bachelor’s in Data Science, where we built a heuristic Snake agent in the course *Outils informatiques pour les sciences* (supervised by Valerio Schiavoni). We now ask whether RL can find better strategies, and how different algorithms compare as the environment gets harder.

Our goal is to benchmark three RL algorithms and study how performance changes when we vary the reward function, the grid size, and the presence of obstacles. The full source code is available at <https://github.com/veborito/RL-project> [11].

2 Problem Formulation

2.1 MDP Definition

We model the game as an MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$.

The state space $\mathcal{S}$ is an 11-dimensional binary vector:

$$s = [\underbrace{d_{\text{straight}}, d_{\text{right}}, d_{\text{left}}}_{\text{danger}}, \underbrace{\text{dir}_{\uparrow}, \text{dir}_{\rightarrow}, \text{dir}_{\downarrow}, \text{dir}_{\leftarrow}}_{\text{heading}}, \underbrace{f_{\uparrow}, f_{\downarrow}, f_{\leftarrow}, f_{\rightarrow}}_{\text{food direction}}]$$

Each bit is 0 or 1. The three danger bits indicate whether moving straight, right, or left from the current position would cause a collision (wall, body, or obstacle). This gives $|\mathcal{S}| = 2^{11} = 2,048$ possible states, which is small enough for a tabular method.

The action space is $\mathcal{A} = \{\text{UP}, \text{DOWN}, \text{LEFT}, \text{RIGHT}\}$. The transition function $P(s' | s, a)$ is deterministic: the only randomness comes from where food respawns. We set $\gamma = 0.99$, which gives enough weight to future rewards to encourage survival while still converging.

We define the optimal policy π^* as the one that maximises the expected discounted return $\mathbb{E}_{\pi}[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)]$. In practice, we evaluate policies by their mean score (food collected per episode) under greedy play, since this directly reflects the game objective.

2.2 Reward Function Design

We compare two reward designs. The sparse reward is:

$$R(s, a) = \begin{cases} +10 & \text{food eaten} \\ -10 & \text{agent dies} \\ 0 & \text{otherwise} \end{cases}$$

The dense reward adds a shaping term at every step:

$$R(s, a) = \begin{cases} +10 & \text{food eaten} \\ -10 & \text{agent dies} \\ +0.1 & \text{moved closer to food} \\ -0.15 & \text{moved away from food} \\ -0.1 & \text{per step (living cost)} \end{cases}$$

The asymmetry (+0.1 vs -0.15) penalises moving away from food more than it rewards getting closer, which is intended to discourage oscillation near food. One standard assumption in reward shaping is that it does not change the optimal policy [14]. Our results show this assumption is violated for PPO, which is discussed in Section 7.1.

3 Environment Implementation

3.1 Gymnasium Wrapper

We implemented two Snake environments following the `Gymnasium` [6] API. The first (`snake_game.py`) is the standard game without obstacles. The second (`snake_w_obstacles.py`) adds k randomly placed obstacles that are regenerated at every `reset()` call, meaning obstacle positions change between episodes. This tests whether a policy trained without obstacles can handle new configurations it has never seen. We use $k = 4$ obstacles in all experiments. The base game logic was adapted from [7].

A key implementation detail is that the danger features in the observation vector are computed by `_is_collision()`, which checks walls, the snake body, and obstacles in the same function. This means that even models trained without obstacles will correctly encode obstacle danger at evaluation time, since the observation construction handles it automatically.

Each `reset()` call accepts a seed argument, which is passed to NumPy to fix food and obstacle placement. This is what allows us to run reproducible evaluations across different seeds. During evaluation, we cap episodes at 500 steps to prevent looping policies from running indefinitely.

4 Algorithms and Implementation

4.1 Random Baseline

The random agent picks one of the four actions uniformly at each step. It serves as a lower bound and is a sanity check: a correctly implemented environment should give a near-zero score to a random agent.

4.2 Tabular Q-Learning

Q-Learning [2] learns a table $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ by applying the Bellman update after each step:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

For this to work well, we need: (1) the state space to be enumerable, which is satisfied since $|\mathcal{S}| = 2,048$; (2) all state-action pairs to be visited often enough, which is promoted by ε -greedy exploration but not strictly guaranteed — in practice, some rare states may be visited infrequently; (3) a decreasing learning rate satisfying the Robbins-Monro conditions $\sum \alpha_t = \infty$ and $\sum \alpha_t^2 < \infty$ to ensure convergence. Our two-phase schedule (constant then reduced) approximates this but does not strictly satisfy these conditions, which means convergence to the true Q-function is not guaranteed. In practice, the Q-table stabilises well before the end of training, suggesting this is not a limiting factor on our environment.

The table is initialised to zeros. We convert the 11-bit observation to a decimal index using a binary-to-integer conversion (`reset_bin` / `step_bin`). Exploration uses ε decaying linearly from 1 to 0 over the first 5,000 episodes. Once $\varepsilon = 0$, we reduce α from 0.1 to 10^{-4} to stabilise the Q-values in the exploitation phase. This two-phase design was motivated by observing instability in Q-values when continuing large updates after convergence.

Hyperparameters (best of 2k vs. 10k episode budgets): $\alpha_0 = 0.1$, $\alpha_{\text{final}} = 10^{-4}$, $\gamma = 0.99$, 10,000 episodes, max 10,000 steps per episode. Implementation adapted from [9].

4.3 Deep Q-Network (DQN-MLP)

DQN [3] replaces the Q-table with a neural network $Q_\theta(s, a)$ trained to minimise the squared Bellman error:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a') - Q_\theta(s, a) \right)^2 \right]$$

where $\bar{\theta}$ are the target network weights and $\mathcal{D}$ is the replay buffer. For this to work well, we assume: (1) the network is expressive enough to approximate the Q-function, which is likely for a 3-layer MLP on 11 inputs; (2) the replay buffer provides approximately

independent samples, which holds when the buffer is large; (3) the target network changes slowly enough to give stable training targets.

The network is: Linear(11 $\rightarrow$ 128) $\rightarrow$ ReLU $\rightarrow$ Linear(128 $\rightarrow$ 128) $\rightarrow$ ReLU $\rightarrow$ Linear(128 $\rightarrow$ 4).

We use a replay buffer of size 10,000 with mini-batches of 512. The target network is updated via soft updates $\bar{\theta} \leftarrow \tau\theta + (1 - \tau)\bar{\theta}$ with $\tau = 0.01$ at every step — this is slower than a hard reset but more stable. We start sampling from the buffer only after 1,000 warmup steps to ensure sufficient diversity. Exploration uses a fixed $\varepsilon = 0.05$ (no decay), which we chose because the network converges quickly and early stopping of exploration would limit performance.

Hyperparameters (best of 2k vs. 10k episode budgets): Adam, lr = 5×10^{-4} , $\gamma = 0.99$, batch 512, warmup 1,000, $\tau = 0.01$, $\varepsilon = 0.05$, 10,000 episodes. Implementation adapted from [10].

4.4 Proximal Policy Optimization (PPO)

PPO [4] is an on-policy actor-critic algorithm that updates the policy by maximising a clipped objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right]$$

where $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ is the ratio between the new and old policy, and $\hat{A}_t$ is the Generalised Advantage Estimate (GAE). The clipping prevents the policy from changing too much in a single update, which makes training stable. An entropy bonus term encourages the agent to keep exploring throughout training rather than converging prematurely to a deterministic policy.

We use the **Stable-Baselines3** [5] implementation with a shared 2-layer MLP [128, 128] for both the policy and value function. The entropy coefficient of 0.01 plays an important role: it is what produces the cautious, long-surviving behaviour we observe in PPO sparse, and it also explains why dense reward hurts PPO (discussed in Section 7.1).

Hyperparameters (best of 200k vs. 500k timestep budgets): lr = 3×10^{-4} , $\gamma = 0.99$, GAE $\lambda = 0.95$, clip $\varepsilon = 0.2$, entropy coef = 0.01, $n_steps = 2,048$, batch 64, $n_epochs = 10$, 500,000 timesteps. Implementation adapted from [8].

In all cases, the reported hyperparameters are the best configuration found after preliminary runs. Beyond the training budget (2k vs. 10k episodes for Q-Learning and DQN-MLP; 200k vs. 500k timesteps for PPO), other hyperparameters such as the learning rate, batch size, and exploration schedule were also tuned based on training curves. The values reported in each algorithm section are those that produced the best and most stable training scores, averaged over the last 500 episodes. The evaluation seeds were kept separate from the selection process to avoid any bias in the final results.

5 Experiment Design

5.1 Evaluation Protocol

All algorithms are evaluated in greedy mode: the agent always picks the action with the highest Q-value (or policy probability), with no exploration. This isolates the quality of the learned policy from random noise.

We evaluate each condition over 100 episodes per seed, across 5 fixed seeds {42, 123, 456, 789, 999}, giving 500 evaluation episodes per condition. The same seeds are used for all algorithms. Episodes are capped at 500 steps to avoid infinite loops. Results are reported as mean with bootstrapped 95% confidence intervals (1,000 resamples). We use bootstrapping rather than standard errors because score distributions are skewed and do not follow a normal distribution.

5.2 Metrics

The primary metric is mean score (food collected per episode), since it directly measures the game objective. We also report the maximum score in a single episode and mean survival time (steps per episode). A high survival with a low score can indicate looping behaviour rather than good performance.

5.3 Reproducibility

All experiments can be reproduced by running `eval.py` from the repository root. The script loads trained model weights from fixed paths, sets the NumPy and PyTorch random seeds at the start of each evaluation, and saves all results as JSON files in `results/final/`. The five evaluation seeds {42, 123, 456, 789, 999} are hardcoded in the script and applied identically to all algorithms. Trained model weights for all configurations (sparse and dense, for Q-Learning, DQN-MLP, and PPO) are versioned in the repository. A full reproduction of all three experiments takes approximately 20–30 minutes on a standard laptop CPU.

To reproduce a specific experiment:

```
python eval.py --mode all --episodes 100
```

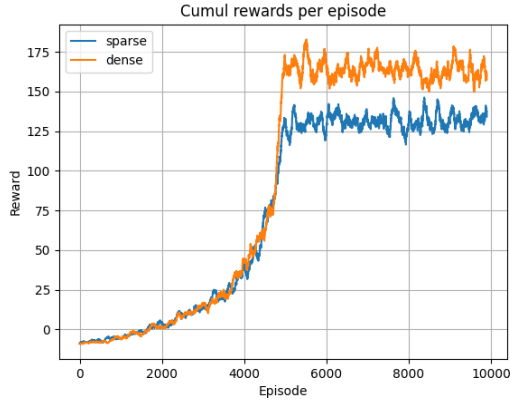
All dependencies are pinned in `pyproject.toml`. The full source code and instructions are available at <https://github.com/veborito/RL-project> [11].

We run three experiments. First, we compare sparse and dense reward on an 8×8 grid with no obstacles (Section 7.1). Second, we test generalisation to larger grids (8×8 , 10×10 , 15×15 , 20×20) with sparse reward and no obstacles (Section 7.2). Third, we compare 0 vs. 4 random obstacles on 8×8 with sparse reward (Section 7.3).

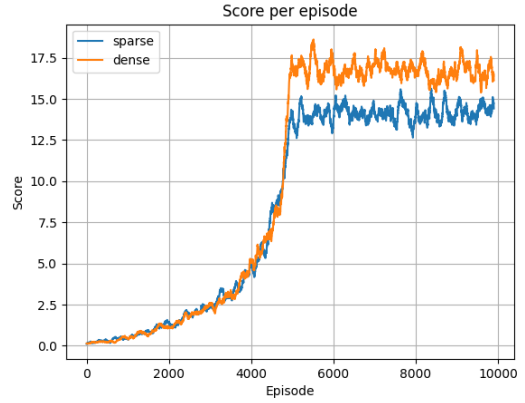
An important caveat applies to experiments 2 and 3: all models are trained on 8×8 without obstacles. These experiments therefore measure zero-shot generalisation, not in-distribution performance.

6 Training

6.1 Q-Learning



(a) Cumulative reward per episode



(b) Score per episode

Figure 1: Q-Learning training over 10,000 episodes. Both variants show a sharp jump around episode 5,000 when $\varepsilon \rightarrow 0$. Dense reaches a higher plateau (≈ 17) than sparse (≈ 14).

The Q-Learning curves have a clear two-phase shape. In the first 5,000 episodes, ε is still positive and the agent explores, scoring little. Once $\varepsilon = 0$, the agent switches fully to greedy play and performance jumps sharply. The dense reward leads to a higher plateau because the distance-based feedback gives the Q-table more signal to work with during learning.

6.2 DQN-MLP

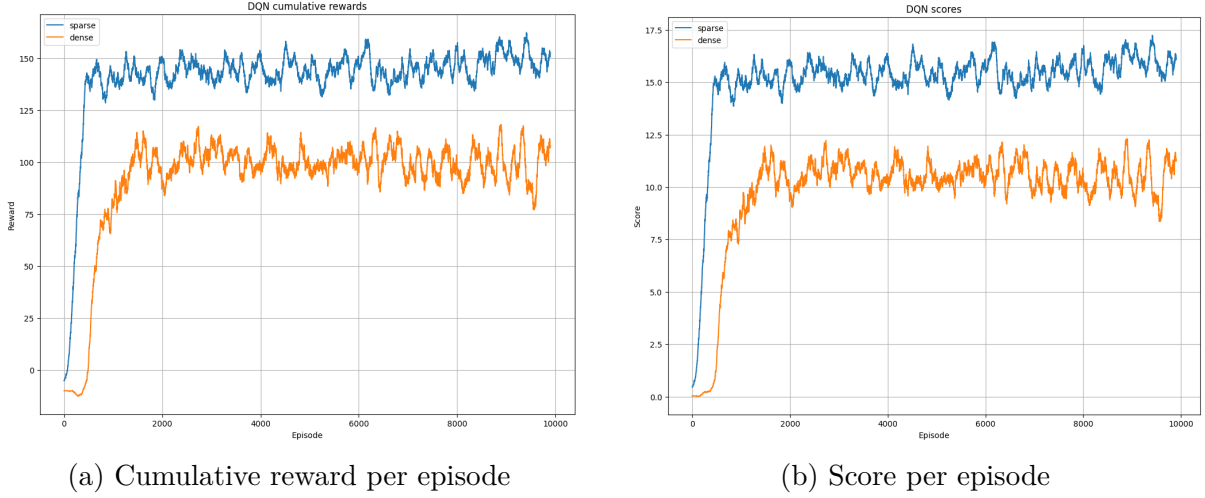
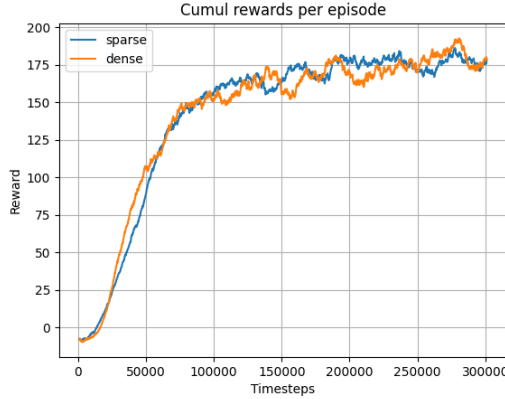


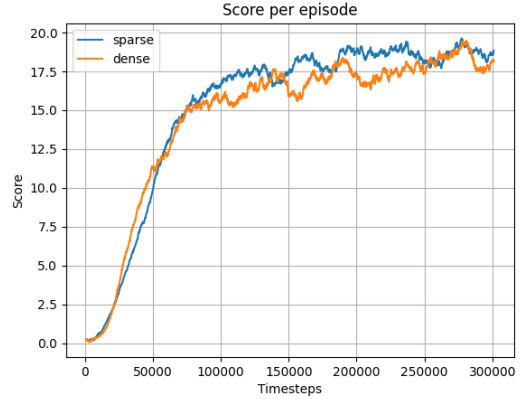
Figure 2: DQN-MLP training over 10,000 episodes. Sparse converges about 3 times faster (~ 500 episodes) than dense ($\sim 1,500$ episodes). Despite lower training scores, the dense model achieves higher greedy evaluation scores.

The most notable observation here is that sparse reward converges about three times faster than dense. This is because dense reward starts with a large negative living cost that penalises all actions equally early in training, which slows learning. The training score of dense (≈ 10 – 11) also appears lower than sparse (≈ 15 – 16), yet at evaluation the dense model performs better (Table 2). This discrepancy is worth examining carefully. During training, scores are computed with $\varepsilon = 0.05$ exploration: at each step, 5% of actions are random, which introduces noise that systematically lowers the observed score. More importantly, the cumulative reward scale differs between sparse and dense — the dense reward accumulates small ± 0.1 shaping terms at every step, so a high cumulative reward does not necessarily mean many food items were collected. When we evaluate greedily and measure only food collected (score), the dense model’s policy turns out to be better calibrated for food-seeking, which is what matters for the final metric.

6.3 PPO



(a) Cumulative reward per episode



(b) Score per episode

Figure 3: PPO training over 500,000 timesteps. Both variants converge quickly ($\sim 50,000$ timesteps) and are nearly indistinguishable during training. The difference between sparse and dense only appears at greedy evaluation.

PPO converges the fastest in relative terms: both sparse and dense reach near-asymptotic performance within about 50,000 timesteps, or 17% of the total training budget. The two training curves are almost identical, which suggests PPO’s on-policy updates are not very sensitive to reward shaping during training. The difference between them only becomes visible when we evaluate with a deterministic greedy policy.

6.4 Convergence Summary

Table 1: Approximate convergence point for each configuration.

Algorithm	Reward	Convergence	% of budget
Q-Learning	Sparse	$\sim 5,000$ ep	50%
Q-Learning	Dense	$\sim 5,000$ ep	50%
DQN-MLP	Sparse	~ 500 ep	5%
DQN-MLP	Dense	$\sim 1,500$ ep	15%
PPO	Sparse	$\sim 50,000$ ts	17%
PPO	Dense	$\sim 50,000$ ts	17%

DQN-MLP sparse is the most sample-efficient algorithm: it uses only 5% of its training budget to converge. Q-Learning needs the full ϵ -decay schedule, which takes up 50% of training. PPO converges at a moderate speed but produces the best final performance.

7 Results

7.1 Sparse vs. Dense Reward

Table 2: Sparse vs. dense reward on 8×8 , no obstacles. Results over 500 evaluation episodes (100 per seed, 5 seeds). CI = bootstrapped 95% confidence interval.

Algorithm	Reward	Mean Score [95% CI]	Max	Survival
Random	—	0.15 [0.13, 0.17]	3	10.5
Q-Learning	Sparse	14.12 [13.81, 14.42]	33	107.6
Q-Learning	Dense	16.45 [16.14, 16.80]	32	130.7
DQN-MLP	Sparse	17.69 [17.37, 18.05]	39	115.4
DQN-MLP	Dense	19.07 [18.68, 19.43]	40	125.5
PPO	Sparse	21.61 [21.22, 21.99]	41	200.9
PPO	Dense	20.11 [19.73, 20.49]	43	133.2

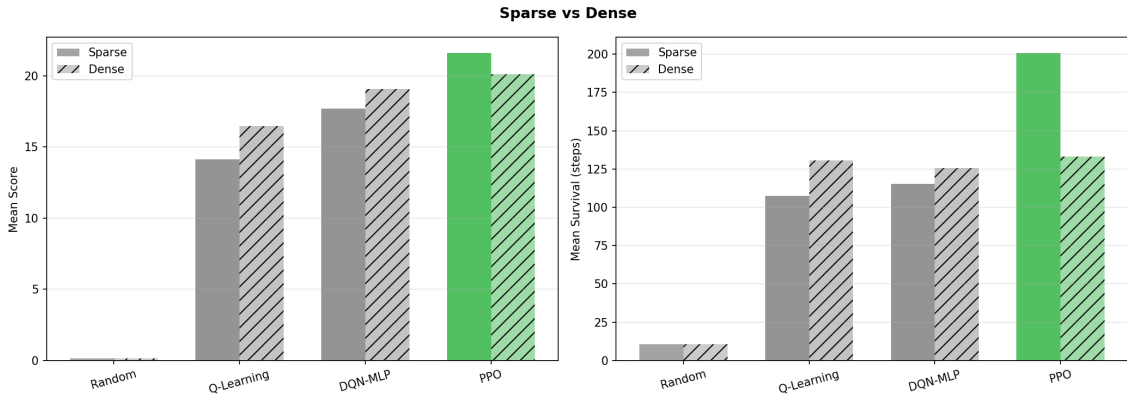


Figure 4: Sparse vs. dense comparison. Left: mean score; right: mean survival. Solid = sparse, hatched = dense.

The performance ranking under sparse reward is as expected: Random < Q-Learning < DQN-MLP < PPO. Dense reward improves Q-Learning by +2.33 points (+16.5%) and DQN-MLP by +1.38 (+7.8%). For both algorithms, the denser feedback helps during training, particularly in early episodes when food is rarely collected and the sparse reward gives almost no signal.

PPO behaves differently: its score drops by 1.50 points (−6.9%) and its survival falls from 200.9 to 133.2 steps (−33.7%) when switching to dense reward. This is the most surprising finding of the paper. The likely explanation is the interaction between the entropy bonus in PPO’s objective and the directional shaping. With sparse reward, the entropy term keeps the policy diverse and exploratory throughout training, which naturally produces cautious, long-surviving agents. The dense reward provides a stronger directional signal

toward food, which overrides this entropy-driven behaviour and pushes the agent to be more aggressive, at the cost of survival. This hypothesis is consistent with the large drop in survival time.

7.2 Grid Size Ablation

Table 3: Mean score by grid size. Sparse reward, no obstacles. All models trained on 8×8 (zero-shot generalisation).

Algorithm	8×8	10×10	15×15	20×20
Random	0.14	0.12	0.10	0.09
Q-Learning	14.30	16.83	22.71	24.35
DQN-MLP	17.70	21.04	27.38	29.21
PPO	21.61	26.17	26.59	20.58

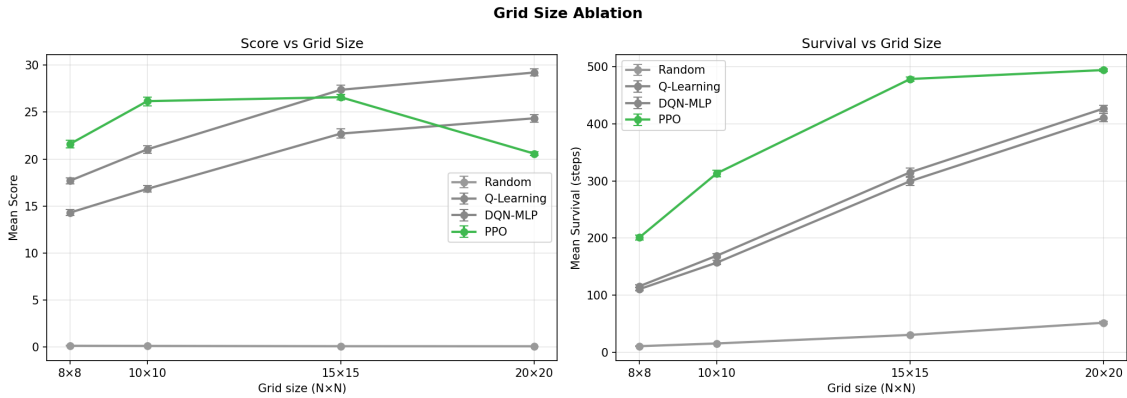


Figure 5: Score and survival as a function of grid size. All models trained on 8×8 .

PPO leads on the two smaller grids but its score peaks at 15×15 and drops at 20×20 , where it is overtaken by both Q-Learning and DQN-MLP. Q-Learning and DQN-MLP, on the other hand, consistently improve as the grid gets larger.

The reason Q-Learning and DQN-MLP generalise better is that the 11-feature observation is grid-size invariant: it encodes relative danger, heading, and food direction, all of which are equally meaningful on any grid size. A policy that avoids nearby danger and moves toward food works the same way on 8×8 or 20×20 . PPO was trained with $n_steps = 2,048$ timesteps per update, which corresponds to many short complete episodes on an 8×8 grid. On a 20×20 grid, episodes are much longer, so PPO’s update schedule sees fewer complete trajectories and the advantage estimates become less accurate. This likely explains why PPO’s score drops while its survival keeps rising (494 steps at 20×20): it can navigate without dying but no longer finds food efficiently.

7.3 Obstacles Ablation

Table 4: Score and survival with 0 vs. 4 random obstacles on 8×8 . Models trained without obstacles.

Algorithm	Mean Score [95% CI]		Survival (steps)	
	No obstacles	4 obstacles	No obstacles	4 obstacles
Random	0.17 [0.15,0.20]	0.10 [0.08,0.12]	10.5	7.4
Q-Learning	14.46 [14.17,14.76]	7.51 [7.18,7.86]	111.1	172.5
DQN-MLP	17.70 [17.36,18.06]	11.26 [10.89,11.63]	115.6	111.5
PPO	21.61 [21.22,21.99]	8.48 [8.05,8.95]	200.9	253.6

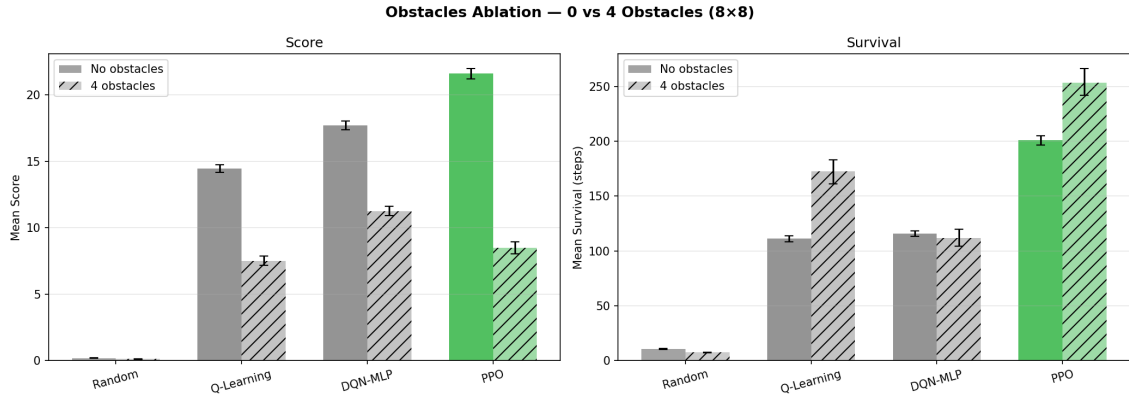


Figure 6: Score and survival with 0 vs. 4 random obstacles. Models trained without obstacles.

All algorithms score less with obstacles. DQN-MLP drops by -36% , which is the smallest reduction. Q-Learning drops by -48% and PPO by -61% , the largest. One notable observation is that Q-Learning and PPO survive *longer* with obstacles ($+55\%$ and $+26\%$ respectively), while DQN-MLP’s survival barely changes (-4%). This is counterintuitive: more obstacles should make it harder to survive, not easier.

The most likely explanation is that obstacles partition the grid into smaller areas, which can trap the agent in loops where it avoids death but never reaches the food. The danger features correctly encode obstacles, so agents know about them, but agents that already prioritise survival (like PPO) may adopt overly cautious avoidance strategies. DQN-MLP, with its lower baseline survival and fixed exploration, maintains more consistent food-seeking behaviour even with obstacles, which explains both its better score retention and unchanged survival.

8 Discussion

On the base setting, PPO is the best algorithm by a clear margin, both in score and survival. Its entropy regularisation leads to a policy that is more cautious and diverse than what DQN-MLP or Q-Learning learn, and this translates into higher performance. The gap between Q-Learning and DQN-MLP is also substantial, which confirms that neural function approximation adds real value even when the observation space is the same.

The effect of dense reward is more nuanced than we expected. It helps Q-Learning and DQN-MLP because they rely on the reward signal to learn directly from transitions, and denser feedback gives them more to work with. For PPO, however, the directional shaping conflicts with the entropy bonus, producing a more aggressive but shorter-lived policy. This suggests that reward shaping is not universally beneficial and its effect depends on the inductive biases of the algorithm.

The grid size results are perhaps the most surprising. We expected PPO to generalise better to larger grids because it has a more expressive policy representation. Instead, Q-Learning and DQN-MLP generalise better, and they overtake PPO at 20×20 . The key factor is that the 11-feature observation is inherently grid-size invariant, so the policies of Q-Learning and DQN-MLP transfer naturally. PPO’s on-policy update mechanism, which is tuned to the episode length distribution of 8×8 , adapts less well to larger grids. This is a limitation worth addressing in future work by either retraining on each grid size or using a larger n_steps value.

The obstacle results show that DQN-MLP is the most robust algorithm in terms of score retention. The survival paradox observed for Q-Learning and PPO highlights an important point: survival time is not a reliable proxy for performance. An agent can survive a long time by looping without food, and this should not be confused with good behaviour.

Several limitations apply to our results. First, all grid size and obstacle experiments use models trained on 8×8 without obstacles, so they measure zero-shot generalisation rather than optimal in-distribution performance. Second, our hyperparameter search was not exhaustive; a proper grid search might improve all baselines. Third, the 500-step episode cap introduces a truncation bias that may underestimate the survival of looping policies. Finally, we did not test the DQN-CNN variant due to a training issue (the model learned to survive but not to eat food), which is left for future work.

9 Conclusion

We compared Q-Learning, DQN-MLP, and PPO on the Snake game across three ablation dimensions. PPO is the strongest algorithm on the base setting, but Q-Learning and DQN-MLP generalise better to larger grids, and DQN-MLP is the most robust to obstacles. Dense reward helps Q-Learning and DQN-MLP but hurts PPO, which shows that reward shaping is not universally beneficial and its effect depends on the algorithm’s inductive biases.

More broadly, these results illustrate that algorithm performance is not monotone in complexity. A more expressive policy (PPO) does not always win, and the choice of algorithm should depend on the specific environment setting — in particular on whether the environment is stationary, whether the state representation is grid-size invariant, and whether exploration and survival are competing objectives. The 11-feature binary observation used here is simple but effective precisely because it is invariant to grid size, which is what allows Q-Learning and DQN-MLP to transfer to larger grids without retraining.

For future work, the most valuable next step would be to retrain each algorithm on each grid size and obstacle configuration for a proper in-distribution comparison. Another direction would be to investigate reward shaping designs that do not conflict with PPO’s entropy bonus, for example potential-based shaping [14] which provably preserves the optimal policy. Finally, extending this study to a visual input (DQN-CNN) would add an interesting comparison point between hand-crafted and learned state representations.

10 Use of AI Tools

Claude (Anthropic) [12] was used to assist with code structure, debugging, implementation of the evaluation pipeline (`eval.py`), and parts of the $\text{\LaTeX}$ report. Gemini (Google DeepMind) [13] was used for specific code generation tasks, in particular the colour palette encoding of grid states. All generated code and text was reviewed and validated by the authors. The algorithmic logic, experimental design, and analysis are the authors’ own work.

References

- [1] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd ed., 2018.
- [2] C.J.C.H. Watkins and P. Dayan. Q-learning. *Machine Learning*, 8(3-4):279–292, 1992.
- [3] V. Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [5] A. Raffin et al. Stable-baselines3: Reliable reinforcement learning implementations. *JMLR*, 22(268):1–8, 2021.
- [6] M. Towers et al. Gymnasium. <https://github.com/Farama-Foundation/Gymnasium>, 2023.
- [7] J. Code. rl_snake: Snake game environment for reinforcement learning. https://github.com/johnnycode8/rl_snake, 2023.
- [8] J. Code. PPO training script for Snake. https://github.com/johnnycode8/rl_snake/blob/master/train_snake.py, 2023.
- [9] Farama Foundation. Training an agent — Gymnasium documentation. https://gymnasium.farama.org/introduction/train_agent/, 2023.
- [10] C. Dimitrakakis. RLDMUU Lab 8: Deep Q-Network. <https://github.com/olethrosdc/rldmuu/blob/main/src/labs/lab8.ipynb>, 2024.
- [11] B. Verdecia Echarte and A. Theiventhiram. RL-project: Source code. <https://github.com/veborito/RL-project>, 2026.
- [12] Anthropic. Claude (AI assistant). <https://www.anthropic.com/claude>, 2024.
- [13] Google DeepMind. Gemini (AI assistant). <https://gemini.google.com>, 2024.
- [14] A. Ng, D. Harada, and S. Russell. Policy invariance under reward transformations. In *Proceedings of ICML*, 1999.